

Welcome!

Welcome to the Code Agent Task Authoring Project! You're stepping into a highly specialized, high-impact initiative designed to improve code agents' abilities to complete common software engineering tasks.

The project runs in two stages:

1. **Pre-work**, where you author coding tasks;
2. **Evaluation sprints** that come afterward, where those tasks are used to run and compare models.

This document is focused on the pre-work stage.

Before completing any tasks, you're required to read this full instructions document to understand the project and your role.

What is this project about?

In this project, you will author code tasks – a real codebase, realistic opening prompt, and short analysis of why a task is hard for a model – that will eventually be used to evaluate model performance. This phase exists for two reasons:

1. Model evaluations are only as good as the tasks they are run on
2. Creating quality prompts during 24-hour sprints discourages iteration

What you produce	A coding-task spec: a real codebase, a realistic opening prompt, and an analysis of why the task is hard.

How it gets used	Later, two coding agents attempt your task and a reviewer compares them. Good tasks are what make that comparison meaningful.
What it is not	You do not run, compare, or rate models in this phase.

This phase will be task authoring only, you will not run, compare, or rate any models here. Where this document mentions working with a model or rating output, it is describing how your task gets used later, not something you do now.

This data will be instrumental in training and improving a model that is used by millions around the world. For this reason, **experts on this project will be held to high quality standards at all times.**

How is this project different from previous Code Preferences sprints?

This project will be **entirely focused on enterprise use cases** for software development. While previous iterations of this project accepted tasks related to a broad range of software, most commonly including popular libraries, upcoming iterations will focus specifically on enterprise-grade software.

Dimension	Previous Sprints	Sprints Going Forward
Repository	Large open source repos, with a focus on enterprise-level software	Only enterprise-level software, including open-source, long-horizon projects, or clones

Task Distribution	Roughly even split between submitted front-end and back-end tasks	Significant push towards back-end tasks ; front-end tasks only accepted if clearly enterprise-grade
Greenfield Development	Greenfield tasks accepted, driving tasks in established repos	Greenfield tasks discouraged , unless genuinely ambitious build-from-scratch (e.g., new service, new library)

Phase 0: Get Set Up

By the end of Phase 0, you'll have Slack, Insightful, Studio, and Training in place, and you'll know where to find tutorial videos.

Steps 1 through 4 are required before your first task, and Step 4 is a resource you'll lean on while you work, so watch it now and come back to it whenever you're in Studio.

1. **Join Slack.** Join the project workplace through Okta on your contract, and set your display name to {First Name} {Last Name}
2. **Set up Insightful to track your time.** [How to use Insightful](#)
 - a. Guide for downloading insightful
 - b. *Important note for Linux users:* Insightful only supports Xorg or X11 Display Servers. See [Linux Installation Guide](#)
3. **Complete pre-work WorkRamp.** Complete the Workramp assessment (look for an email from mail@workramp.com) before you start working.
4. **Complete the pre-work task and get it approved.** Tasks are authored in Studio, and you're auto-provisioned as soon as you complete the Workramp Assessment.
5. **Watch the Studio workflow walkthroughs.** These walk through the Studio environment, task creation, and AutoQC validation. Treat them as a source of truth during pre-work and refer back to them at any time.
 - a. [Code Prefs -- Accessing Studio.mp4](#)
 - b. [Code Prefs -- Creating Tasks.mp4](#)
 - c. [Code Prefs -- Using AutoQC.mp4](#)
6. **Complete sprint work ramp.** Finish the final WorkRamp assessment before joining the project full time.
7. **You're a Writer! Be added to the main project and wait for sprint announcements.** Payment: \$400 for completing onboarding, and your contract

converts to hourly for the sprint.

Note: The onboarding process (with one accepted task) is expected to be completed in one week's time since onboarding to join the main project.

Phase 1: Understand the Project

By the end of this phase you will know what the project is trying to accomplish, the two ways a task can run, and how to classify a task by domain and type. This classification will be used on every task you author

Goal of the project

The project compares how two coding agents handle real software-engineering work. In a later phase your tasks are run through agents, and you'll rate performance, including any issues like agentic laziness or reward hacking. Each sprint will evaluate a variety of models, and your tasks will be seeded with different models every sprint.

Pre-work is the stage you're in now, and it happens before the sprints: here you build the tasks that the later comparison sprints will run and rate models on. A good task is a real challenge, gap, or problem an engineer would actually work on, and it has to be something that current models struggle with. The work is in finding those problems and the codebases where they play out.

As noted above, you are not running or rating anything here; the rest of this section describes the shape of a task so you can author it well.

How a task runs: Interaction Mode

Each task is classified under one of two interaction models, which describe how a model would interact with the prompt when called.

1. **Interaction / pairing:** In the sprint, you will work with the model over several turns, giving feedback, redirecting it, and asking follow-up questions. Design the task so that, when it is later run, it needs 5 or more meaningful turns and roughly 2.5 hours of interaction. Give a clear task-level goal and keep the starting prompt less prescriptive.
2. **Async / autonomous:** In the sprint, the model is given a detailed prompt with all the context, acceptance criteria, and verification steps up front, then runs on its own to the end, and the output is reviewed and rated once it finishes. Tasks in this mode should be:

- a. Harder, longer-running work that would take a person hours to do.
- b. Self-contained, with all relevant details, acceptance criteria, and verification steps stated up front.
- c. Left uninterrupted by you while the model is executing.
- d. Complex enough that a frontier model would run for at least 15 minutes autonomously.

A realistic prompt for an interactive task is very different from one for an asynchronous task; interactive prompts should generally start out much shorter.

Classify the task: Domain

The layer of the stack the task mainly touches. Many real tasks span more than one (a full-stack feature is both frontend and backend), so tag all that apply.

Domain	Target mix	What it covers
Backend	45%	Server-side and infrastructure: API design (REST, GraphQL), database schema and queries, auth, data pipelines, caching, message queues, CLI tools, systems programming, server config, performance work.
Frontend	15%	Client-side and UI: components, state management, styling and responsive design, routing, forms, browser APIs, frameworks (React, Next.js, Vue, Svelte), CSS/Tailwind, build tooling (Vite, Webpack), accessibility, e2e testing.
DevOps/SRE	15%	CI/CD pipelines, terraform/K8s, incident response, observability tooling.
MLE/Data	10%	Eval harnesses, tensor ops, notebooks, rigorous statistical analysis, building and managing data pipelines.

Other	15%	Cross-cutting or specialized work beyond the above. Label what kind of "Other" work it involves.
--------------	-----	--

Classify the task: Task Type

The kind of engineering work involved. Pick the one primary type.

Task type	Target mix	What it is
Debugging	18%	Diagnosing and fixing bugs, error resolution, root cause analysis.
Deployment / DevOps	18%	CI/CD, containerization, infrastructure setup, deployment config.
Code Review	15%	Reviewing PRs, analyzing code quality, giving technical feedback.
Feature Development	9%	Building new functionality or adding features to an existing codebase.
Product Interaction	9%	Visual/UX fixes, UI polish, accessibility, building web apps and interacting with the result.
Refactoring	6%	Restructuring existing code without changing behavior.
Testing	6%	Writing tests, test strategy, improving coverage.
System Design	5%	Planning architecture, designing components, technical decision-making.
Requirement Scoping	3%	Scoping out new requirements for a project.

Migration	2%	Dependency upgrades, framework migrations, language-version upgrades, translating between languages.
Other	9%	Describe in comments.

The percentages are the target mix across all tasks, not a rule for any one task. They show where the project needs volume, so favor the larger buckets when you have a genuine choice.

Example tasks (calibration, not exhaustive)

These are some examples of tasks we're after – don't copy these, use them to gauge scope and difficulty.

- Debug a data pipeline failure involving dependency conflicts and performance bottlenecks across multiple services
- Refactor a large React codebase from CSS modules to Tailwind while preserving visual parity
- Set up a Kubernetes deployment with CI/CD pipeline, including debugging configuration issues
- Plan and implement a cross-service integration with schema migrations and backwards compatibility
- Diagnose and fix concurrency bugs in a Go or Rust service under load
- Review and modernize a legacy codebase: identify architectural issues, propose a migration plan, and begin implementation
- Resolve package migration conflicts when upgrading major framework versions across a monorepo

Every task should also meet these bars:

1. **Commercial realism:** it reflects an actual production development scenario.
2. **Context:** a large codebase (200+ code files) with complex dependencies.
3. **Difficulty:** staff or senior engineer-level work at a large enterprise company, the kind you'd assign to an L5+ SWE. Push for harder, more complex scenarios.

Compensation

The onboarding process will have task-based pay that will be paid out on approval of your first pre-work task for time spent authoring the task, and a task should take about an hour.

1. **Track all your time in Insightful.** It is required on every task, even though pay is not based on the tracking.
2. **We monitor quality, throughput, and time logs for fraud. Unapproved use of GenAI (for example, using an LLM to generate responses) or other fraudulent activity can lead to removal from the project.**

Phase 2: Choose a Repo and Scope the Task

By the end of Phase 2, you'll have a public GitHub repository that meets the project bar, and the task type you're going to build around it.

Picking a codebase

You should start by picking a public GitHub repository that you will use throughout your workflow. The repo needs to be:

1. **Public and on GitHub.** The task against this same repo later, and you'll submit its name and URL, so it has to be reachable.
2. **Large and complex (200+ code files).** Difficulty will come from large, interconnected systems, not a single file change.
3. **Enterprise-grade.** The code should look like a production system a company maintains, not a tutorial. Beginner-level gaps that mature teams have already solved won't surface useful behavior.

Update: An invalid repo breakdown has been added below. Please reference it!

In general, we're aiming for clones of enterprise products or established enterprise-grade open-source projects. Two options tend to be good starting points;

1. An open-source repo you know well. A great option is using a familiar codebase, as you'll later have to explain what the codebase does.
2. A large personal project (200+ files). Allowed if you consent to it being used in LLM training. It must be public on GitHub, have an extended commit history, and not be primarily AI-generated code.

Before committing, check the [restricted list](#) and make sure your repo isn't on it – tasks built on these repos will be rejected. The repos below are good examples of repos, but shouldn't be used for your own tasks:

1. [Google Cloud Platform k8s-config-connector](#)
2. [AWS Command Line Interface](#)
3. [Netflix TLS Certificate Creation](#)

Invalid repo categories (quick reference)

Invalid repo category	Examples
Container engines & orchestrators	Kubernetes, Podman
Databases & storage engines	TiKV, CockroachDB
Observability & telemetry pipelines	OpenTelemetry Collector
Developer tooling & emulators	Floci
UI frameworks	Flutter
Utility libraries	NLog, Spring Session

Pick the task type

Choose the single [primary task type](#) that the repo naturally supports. Pick what the codebase genuinely lends itself to, rather than forcing a type on it, since that choice shapes the prompt you write next.

Phase 3: Write the Opening Prompt

By the end of Phase 3 you'll have an opening prompt that reads like real work, fits your interaction mode, and is hard enough to put a model under genuine pressure. The prompt is what the model receives, and it sets the scope and difficulty of everything that follows.

If the prompt is trivial, or already solved in the codebase, it'll get resolved on the first turn, and produces no useful signal for training a model. On the other hand, if the prompt is so broken or underspecified that it can't be completed at all, it produces no useful signal. The useful zone is in between: a non-obvious problem that takes several turns to get right: several meaningful turns in an interactive task, or a long, multi-step

autonomous run in an async one. Often that difficulty comes from the codebase and the setup, not from piling detail into the prompt.

What makes a good starting prompt

1. **Realistic.** It's a request you'd actually send a coding agent during real work.
2. **Non-Trivial Solution.** The work should involve multiple meaningful turns or long-running work that requires many steps for interactive and asynchronous tasks, respectively.
3. **Not artificially detailed.** Don't over-engineer it with unnecessary specs or constraints just to look thorough. Diversity and realism matter more than length.
4. **Success is allowed.** The task doesn't have to be unsolvable. It's fine if it gets solved, as long as the path there is non-trivial and demanding in a way which surfaces substantial failures from the model.

Match the prompt to the interaction mode

1. **Interactive / pairing:** keep it hard enough to take several turns, with the difficulty coming from realistic ambiguity and codebase context rather than a long spec.
 - a. A common failure mode for interactive prompts is over-writing them: long, highly prescriptive prompts full of lists and code snippets that no one would actually send a coworker. If your interactive prompt reads like a spec sheet, trim it. Remove the lists, long code blocks, and overly specific instructions, and let the natural request stand.
2. **Async / autonomous:** make it self-contained, with the goal, acceptance criteria, and verification steps stated up front, but don't over-specify with step-by-step instructions or exhaustive criteria. The model should have enough to run unattended without being walked through every move.

A few examples of iterative prompts, by task type:

1. **DevOps:** Staging deploy is stuck. ArgoCD shows Progressing for 20 min. kubectl access provided. Get it unstuck and tell me why.
2. **Review (400-line diff with 2–3 real issues planted):** Here's a PR adding SSO login. Review it. I'm worried about session handling but look at the whole thing.
3. **Debug (multi-service repo; symptom given, root cause not):** File uploads intermittently fail with 'connection reset' after ~45s, EU only. Logs attached. Figure out what's going on.

Set the high-level task goal:

State explicitly what success looks like for this task, defining what it means for a model to be complete with the prompt. Write the goal so a reviewer can tell whether the task was actually completed.

Worked examples: Good vs. Bad Prompts

These show the reasoning behind good and bad prompts, not templates to copy. A task that closely mirrors one of these examples will be rejected as duplicative.

Example #1: Iterative Prompt (Feature Development)

Good

I need you to create a utility for the Gradient-Constrained Steiner Tree Task in this repository. Please keep the solution deterministic, production-ready, and aligned with existing codebase conventions. We'll also need a plotting tool for debugging.

Explanation: Natural prompt. High-level guidance. The Model is expected to iterate with you to fully implement this solution (Note: As the Expert, you are expected to have a much more detailed understanding in your head. This should be explained in the “Difficulty & Codebase Understanding” section).

Bad

Create a utility for the Gradient-Constrained Steiner Tree Task in this repository. The solution must be fully deterministic, production-ready, aligned with existing conventions, and have 100% code coverage

Model Shapes

...

Code snippet

...

...

Testing

1. ...
2. ...
3. ...

Constraints

1. ...
2. ...
3. ...

Definition of Done

1. ...
2. ...
3. ...

[Further details truncated in this example for brevity]

Explanation: Highly prescriptive prompt, more suitable for Asynchronous tasks. Not representative of a day-to-day interaction with a coding agent, written by a human

Example #2: Iterative Prompt (Debugging)

Good

We're dealing with a major production stability crisis. The whole app is freezing up for several minutes, exactly at 12:00pm every day, right when our "Recipe of the Day" feature updates. Telemetry is showing a significant DB CPU spike right before it happens, which then cascades into request timeouts across the entire application.

Investigate this. Start with the cache regeneration logic. The goal is to ensure that when a hot-ticket item expires, only one process is ever allowed to regenerate it from the database. All other concurrent requests for that same item should wait for the new value to be populated, not hammer the database themselves.

After we fix the issue, in addition to creating a test for this exact issue, we'll need new testing covering high-concurrency scenarios.

Explanation: Bug is explained naturally. Realistic scenario. High-level details provided. Model expected to start investigating and directed to start in a probable area. Important details such as testing are mentioned without being fully explained, signaling the Model should circle back after the fix is identified or fixed.

Bad

We have a bug where the app freezes up every day. Something is timing out everything. Examine the whole system end to end and fix this now. Be sure to test everything.

Explanation: Low effort prompt. Non-technical. The scenario is realistic but the data is insufficient. Language like “Examine the whole system” and “test everything” is far too broad-stroke in enterprise-level settings.

Example #3: Async Prompt (Debugging)

Good

Anonymous link survey submissions are not idempotent. If a respondent submits a survey on an unstable connection, the server can successfully create the response but the client may time out before receiving the result. When the SDK retries, Formbricks can create a second Response record for the same submission attempt. This can also duplicate downstream side effects such as webhooks, integrations, quota evaluation, and follow-up emails.

Please investigate the anonymous link survey submission flow end-to-end and implement a production-ready idempotency fix.

The solution should ensure that retries for the same submission attempt return or reuse the original response instead of creating a duplicate response or re-triggering downstream side effects. At the same time, anonymous link surveys must still allow multiple intentional submissions from different attempts.

Please keep the change backward compatible for older clients, avoid changing webhook/pipeline payload contracts unless strictly necessary, and avoid broad unrelated refactors.

Add or update tests that prove:

- retrying the same anonymous link survey submission does not create duplicate Response records
- downstream side effects are not triggered twice for the same submission attempt
- separate intentional anonymous submissions still work
- existing app survey and single-use link behavior is not broken

Run the most relevant test suite and typecheck/lint commands for the changed area.

Explanation: Sufficiently prescriptive. Realistic scenario. Technical details are provided so the model has a solid starting point to begin debugging. Constraints are included, along with testing expectations.

Bad

Anonymous link survey submissions are not idempotent. Trace all code paths for link submissions, identify the bug, and fix it. Retries should be memory efficient. Don't do any additional work. Make the smallest possible change. Be sure to test everything and that all changes are backwards compatible.

Explanation: Not prescriptive enough, more suitable for an iterative task. The basic idea is in place, but the problem and expected solution are far too open-ended for both an agent or a SWE to understand the full scope of the task in one go.

Example #4: Async Prompt (Refactor)

Good

Conduct a code-driven security audit and implement fixes for high and critical severity issues, especially those aligned with OWASP Top 10 (e.g., unsafe deserialization, injection, broken access control, SSRF, misconfiguration).

You must:

- Explore the codebase thoroughly (models, views, tasks, serializers, Celery config, middleware, integrations).
- Trace data flows across trust boundaries (user input → processing → storage → async tasks → output).
- Identify vulnerabilities that could lead to:
 - Remote Code Execution (RCE)
 - Privilege escalation
 - Data leakage or corruption

Requirements:

For each issue:

- Show where it exists in code
- Explain exploit scenario and impact
- Provide a secure fix with code changes
- If insecure serialization exists (e.g., pickle in Celery, caching, or storage):
 - Replace with a safe alternative (JSON or equivalent)
 - Handle non-serializable objects
- Ensure compatibility with existing data and workers

Maintain:

- Backward compatibility is non-negotiable, if not possible - document the possible plan
- API stability
- Safe data migration strategy (if persistence format changes)

Validation:

You must define and/or implement:

- Unit + integration tests for all changes
- Security tests (e.g., malicious payloads for deserialization/injection)
- Regression checks to ensure no functional breakage
- Validation for:
 - Existing data compatibility
 - In-flight Celery tasks
 - Partial/mixed deployments

Deliverables:

- Security audit summary (issues + severity + exploitability)
- Code-level fixes with explanation
- Migration/rollout plan (safe, incremental, zero-downtime where possible)
- Test coverage additions
- Residual risks / limitations

Constraints:

- Do not make assumptions. Always verify in code
- Avoid superficial or generic advice
- Do not introduce breaking changes without mitigation
- Handle edge cases (invalid data, partial migrations, mixed versions)

Success Criteria:

- All critical/high vulnerabilities are addressed with validated fixes
- No unsafe patterns remain in exploitable paths
- System remains functional and backward compatible
- Changes are tested, secure, and production-ready

Explanation: Prescriptive prompt. Realistic scenario for a Refactor as general problem areas are identified. Includes a clear set of expectations detailing the **process** of completing the work along with Deliverables and Success Criteria so the model can establish when the work is considered done.

Bad

Examine the codebase to conduct a code-driven security audit. Pick 3 high-priority fixes for high and critical severity issues. At the end, explain what you fixed and provide a full report to prove why your fixes improve security.

Show all your work. Make all changes backwards compatible. Test everything.

Explanation: Not prescriptive. Somewhat realistic scenario for a Refactor, but far too open-ended for an Asynchronous task. Prompt reads like, “Find 3 things and fix them. Give me a report. Don’t break anything. Show your work. Test everything”. Not enough context given. No clear definition of done or criteria for success.

Phase 4: Write the Rationales

By the end of Phase 4 you'll be able to write the three answers that show the task is genuinely hard and that you understand the codebase.

These three answers are what the reviewer scrutinizes most. They have to show two things: that the task is genuinely difficult, and that you actually understand the code it runs against. Be specific and point at the real mechanism. Generic answers and LLM-generated answers will be rejected.

⚠️ Avoid using tools to improve spelling and grammar, such as Grammarly, Quillbot, PerfectIt, etc. These can trigger LLM flags. Keep all answers human-written, without the use of any tooling.

"Why do you think this is a task that frontier models would struggle with?"

A strong answer names the specific property of the task that makes it hard (a subtle invariant, a concurrency race, a fix that spans several files) rather than asserting difficulty in the abstract.

✗ **Weak:** "It has some bugs."

✓ **Strong:** "The webhook handler isn't idempotent and races on the ledger write under load. Reproducing it means simulating concurrent events, and the fix spans the retry logic and the DB transaction boundary. This represents the kind of on-call debugging a model might struggle with."

"What does this codebase do?"

A strong answer says what the system is, the stack and rough size, and the main modules that matter to the task.

✗ **Weak:** "It's a web app."

✓ **Strong:** "A Python/FastAPI service (~300 files) that ingests payment webhooks and writes to a Postgres ledger, with modules for auth, webhook routing, and a nightly reconciliation job."

"What issues or gaps does the codebase have, and how would a developer get started on solving them?"

A strong answer names the concrete gap and gives a realistic first step a developer would take, usually a failing test that pins the behavior before any fix.

✗ **Weak:** "The code failed, and the developer would look at the docs."

✓ **Strong:** "Events get processed twice on retry. A developer would start by writing a failing test that fires duplicate webhooks, then add an idempotency key on the event ID inside the DB transaction."

Phase 5: QC and Submit

By the end of Phase 5 you'll have run AutoQC, cleared the submission checklist, and sent the task for review.

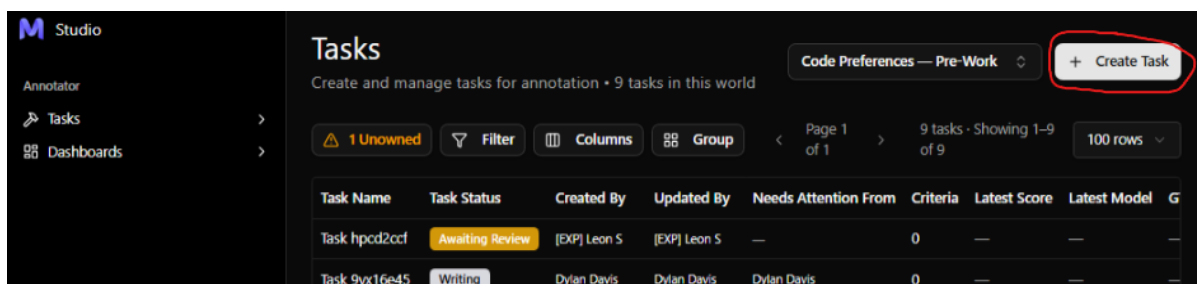
Submit in Studio

You author and submit the task in Studio. The opening prompt (Phase 3) and the three rationales (Phase 4) are the substance; these steps are the mechanics.

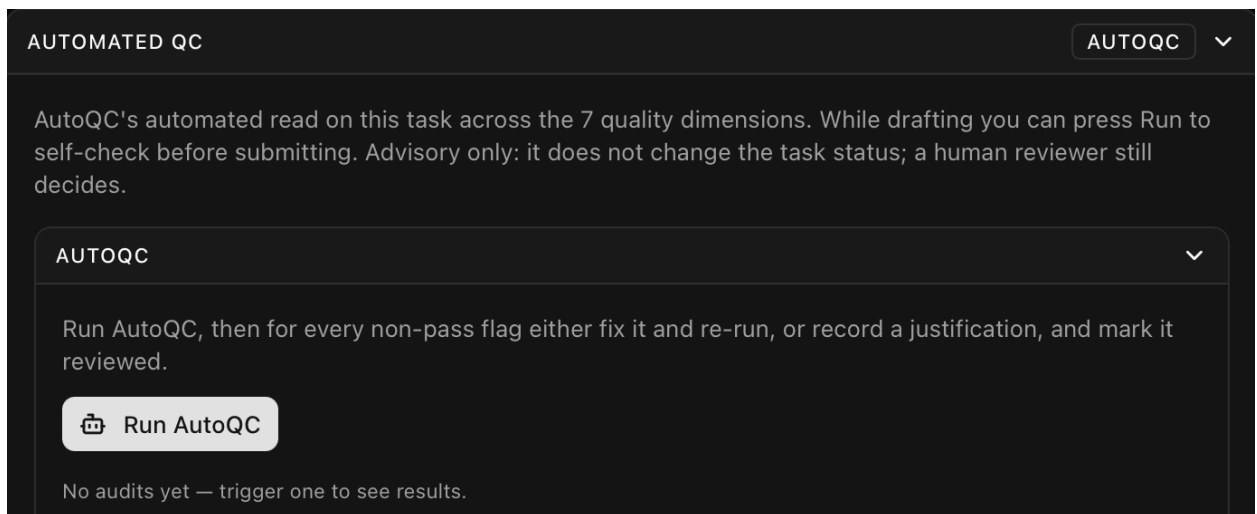
The Studio workflow videos walk through the whole thing end to end, and we have included the general steps below:

- [Code Prefs -- Accessing Studio.mp4](#)
- [Code Prefs -- Creating Tasks.mp4](#)
- [Code Prefs -- Using AutoQC.mp4](#)

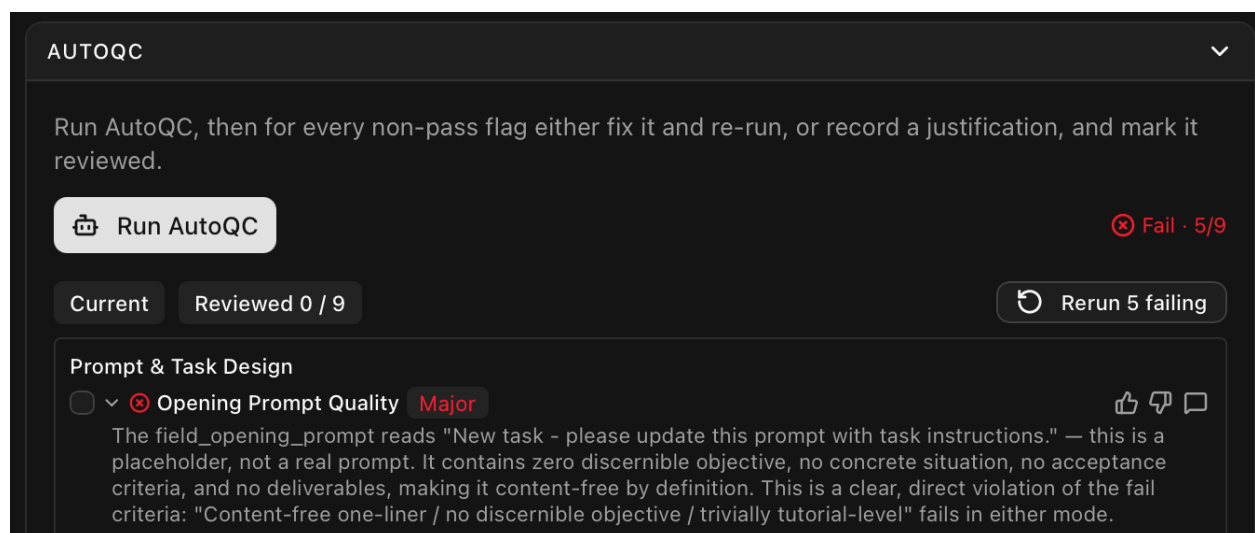
1. **Create the task.** From the Tasks view, click "+ Create Task" at the top right.



2. **Fill the Repository & Tags card.** Add the repo name and public GitHub URL, then set Primary Language, [Task Type](#) (one primary), [Domain](#) (all that apply), and Interaction Mode.
3. **Write the Opening Prompt card.** Covered in Phase 3: write it like a real ticket or Slack message and match it to the interaction mode.
4. **Answer the three Rationale questions.** Covered in Phase 4: be specific, since generic or LLM-generated answers are rejected.
5. **Run AutoQC.** Once every required field is filled, open the Automated QC card and click Run AutoQC.



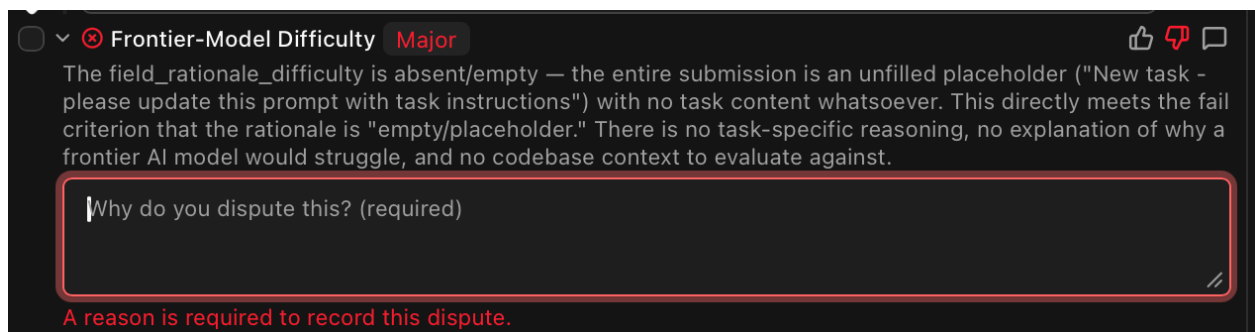
Review the latest result, fix anything that failed, and run it again until it passes. A "Pass" result is still worth reading, because it explains the requirements.



If any AutoQC failure cannot be resolved, you may manually work around the failure after providing an explanation. Every Category has three options located on the right hand: Agree, Dispute, Comment



Choose 👎 (Disagree), then Explain your rationale



The screenshot shows a task review interface. At the top, there's a dropdown menu with a red 'x' icon, followed by the text 'Frontier-Model Difficulty' and a red 'Major' tag. To the right are three icons: a thumbs up, a thumbs down (highlighted in red), and a speech bubble. Below this, a text block reads: 'The field_rationale_difficulty is absent/empty — the entire submission is an unfilled placeholder ("New task - please update this prompt with task instructions") with no task content whatsoever. This directly meets the fail criterion that the rationale is "empty/placeholder." There is no task-specific reasoning, no explanation of why a frontier AI model would struggle, and no codebase context to evaluate against.' Below the text is a large text input field with a red border and a red outline. The placeholder text inside the field is 'Why do you dispute this? (required)'. At the bottom of the field, there's a red error message: 'A reason is required to record this dispute.'

6. **Submit for Review.** Click Submit for Review in the Writer Actions card. Your inputs lock and the task moves to Awaiting Review.
 - a. If the Submit button doesn't work, a required field is empty. An error toast appears at the top, so check all tags, the prompt, and all three rationale answers.

Before you submit: checklist

Run through this before hitting Submit – this checklist is directional, and does not guarantee a pass

Codebase and setup

- ☐ Repo is public on GitHub and not on the restricted list.
- ☐ 200+ code files with real complexity (not a docs-only or .md/.txt-heavy repo).
- ☐ Enterprise-grade codebase, not a toy, tutorial, or unmaintained project.

Task quality

- ☐ Production-realistic scenario a real developer would face.
 - ☐ Concrete goal, not vague (no "make it better" without success criteria).
 - ☐ Touches multiple files (roughly 5 to 15+), not a single-file or tutorial-level change.
 - ☐ Non-obvious: not solvable in one turn and not already solved in the repo.
 - ☐ A senior engineer would plausibly assign it, and it needs genuine problem-solving ☐
- Hard enough to surface meaningful differences when the task is evaluated later.
- ☐ Clear, testable sense of what "done" looks like.

Tags and mode

- ☐ Primary Language, Task Type (one), Domain(s), and Interaction Mode are all set.
- ☐ Task Type matches the actual work (for example, not "Debugging" when the task is really building a feature).
- ☐ Prompt matches the mode: interactive reads natural and less prescriptive; async is self-contained with acceptance criteria and verification.

Rationale

- ☐ All three are answered, specific, and in your own words (not generic, not LLM-generated).

Final gate

- ☐ AutoQC has been run and passes.

If your task comes back for revision

A reviewer can send a task back, and the Writer Feedback will explain what needs to change. Make those changes, re-run AutoQC, and submit again.

FAQ and Support

Task Requirements

Minimum Requirements

- Large codebases (200+ code files) with complex dependencies
- Commercial realism: Tasks should reflect actual production scenarios

- Minimum 3 sentences of thoughtful, human-written feedback per open-ended response

Q: How detailed should prompts be?

A: Prompts should be diverse and similar to how you'd actually use a coding agent—no need to make them artificially detailed. However, the initial prompt should be complex enough to provide a challenging starting point for the task.

Q: Does the prompt need to cause a model to fail?

A: Not necessarily, but tasks should be long and difficult enough to provide a signal on a model. Ideally, these are challenging tasks where the model struggles over multiple turns before succeeding (if it succeeds at all). Your task should always push models so they show behavioral issues nonetheless.

Q: Are there repositories I should avoid?

A: Yes. Check the restricted repository list before starting. Crosscheck and validate that your chosen repository is not on the restricted list. Tasks with repository in the restricted list will be rejected. The link to that list is [here](#).

Access & Onboarding

Q: Can I get access to recording of one of the onboarding sessions?

A: Yes, we will share a link to the onboarding session recording following the meeting.

Q: Do we have to fill the username and email address on the Studio onboarding again if we were part of the previous sprints and have filled it once?

A: Yes, if the onboarding tab is empty do fill it out